

P0152R1 `constexpr atomic<T>::is_always_lock_free`

Author: Olivier Giroux

Contact: ogiroux@nvidia.com

Author: JF Bastien

Contact: jfb@google.com

Author: Jeff Snyder

Contact: jeff-isocpp@caffeinated.me.uk

Date: 2016-03-02

Previous: <http://wg21.link/N4509>

Previous: <http://wg21.link/P0152R0>

URL: <https://github.com/jfbastien/papers/blob/master/source/P0152R1.rst>

Source: <https://github.com/jfbastien/papers/blob/master/source/P0152.cc>

The current design for `std::atomic<T>` affords implementations the critical freedom to revert to critical sections when hardware support for atomic operations does not meet the size or semantic requirements for the associated type `T`. This:

- Preserves C++ support on aging hardware.
- Supports developers who don't target a specific architecture e.g. with the `-march=xxx` flag.
- Improves the portability of abstract representations for C++ programs, e.g. when compiling C++ code to execute portably within a web browser.

The Standard also ensures that developers can be informed of the implementation's lock-freedom guarantees, by using the `is_lock_free()` member and free-functions. This is important because programmers may want to select algorithm implementations, or even select algorithms, based on this knowledge. Developers are equally likely to do so for correctness and performance reasons.

The software design shipped in C++11 and C++14 is, however, somewhat sandbagged.

There is poor support for static determination of lock-freedom guarantees.

At the present time the Standard has limited support in this domain: the `ATOMIC_..._LOCK_FREE` macros that return `2`, `1` or `0` if the corresponding atomic type is always lock-free, sometimes lock-free or never lock-free, respectively. These macros are little more than a consolation prize because they do not work with an arbitrary type `T` (as the C++ native `std::atomic<T>` library intends) and they leave adaptation for generic programming entirely up to the developer.

This leads to the present, counter-intuitive state of the art whereby non-traditional uses of C++ have better support than high-performance computing. We aim to make the smallest possible change that improves the situation for HPC while leaving all other uses untouched.

We propose a `static constexpr` complement of `is_lock_free()` that is suitable for use with SFINAE and `static_assert`.

Proposed addition

Under 29.5 Atomic types [atomics.types.generic]:

```
namespace std {
    template <class T> struct atomic {
        static constexpr bool is_always_lock_free = implementation-defined;
        // Omitting all other members for brevity.
    };
    template <> struct atomic<integral> {
        static constexpr bool is_always_lock_free = implementation-defined;
        // Omitting all other members for brevity.
    };
    template <class T> struct atomic<T*> {
        static constexpr bool is_always_lock_free = implementation-defined;
        // Omitting all other members for brevity.
    };
}
```

Under 29.6.5 Requirements for operations on atomic types [atomics.types.operations.req], between paragraphs 6 and 7:

```
static constexpr bool is_always_lock_free = implementation-defined;
```

The `static` data member `is_always_lock_free` is true if the atomic type's operations are always lock-free, and false otherwise.

[Note: The value of `is_always_lock_free` is consistent with the value of the corresponding `ATOMIC_..._LOCK_FREE` macro, if defined. — end note]

Under 29.6.5 Requirements for operations on atomic types [atomics.types.operations.req], in paragraph 7:

[Note: The return value of the `is_lock_free` member function is consistent with the value of `is_always_lock_free` for the same type.

– end note]

The `__cpp_lib_atomic_is_always_lock_free` feature test macro should be added.

Additional material

We did not provide the `atomic_is_always_lock_free` C-style free functions (which the `is_lock_free` functions have) because these require a pointer. This makes the free functions significantly less useful as compile-time `constexpr`.

We show a sample implementation:

```
namespace std {

    namespace detail {
        // It is implementation-defined what this returns, as long as:
        //
        // if (std::atomic<T>::is_always_lock_free)
        //     assert(std::atomic<T>()::is_lock_free());
        //
        // An implementation may therefore have more variable template
        // specializations than the ones shown below.
        template<typename T> static constexpr bool is_always_lock_free = false;

        // Implementations must match the C ATOMIC_*_LOCK_FREE macro values.
        template<> static constexpr bool is_always_lock_free<bool> = 2 == ATOMIC_BOOL_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<char> = 2 == ATOMIC_CHAR_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<signed char> = 2 == ATOMIC_CHAR_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<unsigned char> = 2 == ATOMIC_CHAR_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<char16_t> = 2 == ATOMIC_CHAR16_T_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<char32_t> = 2 == ATOMIC_CHAR32_T_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<wchar_t> = 2 == ATOMIC_WCHAR_T_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<short> = 2 == ATOMIC_SHORT_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<unsigned short> = 2 == ATOMIC_SHORT_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<int> = 2 == ATOMIC_INT_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<unsigned int> = 2 == ATOMIC_INT_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<long> = 2 == ATOMIC_LONG_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<unsigned long> = 2 == ATOMIC_LONG_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<long long> = 2 == ATOMIC_LLONG_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<unsigned long long> = 2 == ATOMIC_LLONG_LOCK_FREE;
        template<typename T> static constexpr bool is_always_lock_free<T*> = 2 == ATOMIC_POINTER_LOCK_FREE;
        template<> static constexpr bool is_always_lock_free<std::nullptr_t> = 2 == ATOMIC_POINTER_LOCK_FREE;

        // The macros do not support float, double, long double, but C++ does
        // support atomics of these types. An implementation shall ensure that these
        // types, as well as user-defined types, guarantee the above invariant that
        // is_always_lock_free implies is_lock_free for the same type.
    }

    template<typename T>
    struct atomic_n4509 {
        // ...
        static constexpr bool is_always_lock_free = detail::is_always_lock_free<T>;
        // ...
    };

}
```